

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования

**«Национальный исследовательский Нижегородский
государственный университет им. Н.И. Лобачевского»
(ННГУ)**

Институт информационных технологий, математики и механики

Центр прикладных информационных технологий

Направление подготовки: «Фундаментальная информатика и
информационные технологии»

Профиль подготовки: «Инженерия программного обеспечения»

Отчет по лабораторной работе на тему:
«Алгоритм Дейкстры на 3-куче и 15-куче»

Выполнил:

студент группы 3822Б1ФИ1

_____ Чистов А. Д.

Проверил:

Ассистент каф. АГДМ

_____ Уткин Г. В.

Нижний Новгород

2024 г.

Содержание

Введение	2
1 Основные определения	3
2 Математическая постановка задачи	4
3 Описание алгоритмов	5
3.1 Класс DHeap	5
3.2 Алгоритм Дейкстры	7
4 Программная реализация алгоритма	10
4.1 Описание класса DHeap	10
4.2 Описание функций	12
5 Эксперименты	15
Заключение	18
Список литературы	19
Приложения	20

Введение

Одной из центральных задач теории графов является нахождение кратчайших путей в графах, что имеет широкое практическое применение в задачах маршрутизации, логистики, сетевого планирования и моделирования.

Алгоритм Дейкстры, предложенный Эдсгером Дейкстрой в 1959 году, стал классическим подходом для решения этой задачи на графах с ненулевыми положительными весами рёбер [1]. Он позволяет находить кратчайший путь от одной вершины графа до всех остальных, и основой его эффективности является использование структур данных, таких как куча, для оптимизации извлечения минимального значения.

Обычно в алгоритме Дейкстры используется двоичная куча, обеспечивающая хорошую производительность для стандартных задач. Однако для улучшения производительности при работе с большими графами и для различных практических применений может использоваться D-арная куча (например, 3-куча, 15-куча и др.), позволяющая сократить количество операций по сравнению с двоичной кучей за счёт уменьшения высоты дерева. В частности, 3-куча и 15-куча могут быть полезны для ускорения алгоритма в зависимости от структуры графа и особенностей доступа к памяти.

Цели и задачи

1. Исследовать тематическую литературу.
2. Реализовать класс DHeap.
3. Реализовать алгоритм Дейкстры.
4. Провести экспериментальное сравнение производительности и вычислительной эффективности алгоритма на различных типах графов.
5. Построить графики сравнения алгоритмов от числа рёбер и вершин.
6. Сделать выводы о проделанной работе.

1 Основные определения

Введём основные определения, необходимые для формализации задачи:

- Граф $G = (V, E)$ — ориентированный граф, где V — множество вершин ($|V| = n$), а E — множество рёбер.
- Вес ребра $w(u, v)$ для ребра $(u, v) \in E$ определяется функцией $w : E \rightarrow \mathbb{R}_{\geq 0}$, где $w(u, v) \geq 0$ для всех рёбер.
- Кратчайший путь из вершины $s \in V$ в вершину $v \in V$ — это путь $P \subseteq E$, минимизирующий сумму весов рёбер:

$$W(P) = \sum_{(u,v) \in P} w(u, v).$$

- Минимальное расстояние от вершины s до вершины v обозначается $d(s, v)$ и определяется как:

$$d(s, v) = \min_{P \in \mathcal{P}(s, v)} W(P),$$

где $\mathcal{P}(s, v)$ — множество всех путей от s до v .

- Если вершина v недостижима из вершины s , то $d(s, v) = \infty$.

2 Математическая постановка задачи

Алгоритм Дейкстры решает задачу поиска кратчайшего пути из одной вершины $s \in V$ (стартовой) графа $G = (V, E)$ с весами на рёбрах $w : E \rightarrow \mathbb{R}_{\geq 0}$ до всех остальных вершин графа. Пусть:

- V — множество вершин графа ($|V| = n$),
- E — множество рёбер,
- $w(u, v)$ — вес ребра (u, v) , если $(u, v) \in E$, и бесконечность (∞) иначе.

Необходимо найти минимальное расстояние $d(s, v)$ от вершины s до каждой вершины $v \in V$, удовлетворяющее следующим условиям:

$$d(s, s) = 0,$$
$$d(s, v) = \min_{P \in \mathcal{P}(s, v)} \sum_{(u, v) \in P} w(u, v), \quad \forall v \in V,$$

где $\mathcal{P}(s, v)$ — множество всех путей из s в v , а P — последовательность рёбер, соединяющая вершины s и v , минимизирующая суммарный вес.

Алгоритм Дейкстры находит кратчайшие расстояния для всех вершин графа G , начиная с вершины s , и позволяет эффективно вычислять минимальные расстояния до всех остальных вершин.

3 Описание алгоритмов

3.1 Класс DHeap

Класс *DHeap* реализует d -арную кучу, где d — количество дочерних узлов для каждого узла.

- Конструктор принимает целое значение d и проверяет, что оно больше или равно 2. Если $d < 2$, генерируется исключение.

Сложность: $O(1)$ — проверка значения d выполняется за постоянное время.

- **Insert** вставляет новый элемент в кучу. Элемент добавляется в конец вектора, после чего вызывается функция **SiftUp** для поддержания свойств кучи.

Сложность:

- Добавление элемента в конец массива — $O(1)$.
- **SiftUp** имеет сложность $O(\log_d(n))$, так как высота d -арного дерева пропорциональна $\log_d(n)$.

Итоговая сложность: $O(\log_d(n))$.

- **ExtractMin** извлекает минимальный элемент из кучи. Если куча пуста, генерируется исключение. Минимальный элемент сохраняется, последний элемент вектора перемещается на место корня, после чего вызывается **SiftDown**.

```
1 function ExtractMin(var v: array of Integer): Integer;  
2 begin  
3     if IsEmpty(v) then  
4         raise Exception.Create('Heap is empty');  
5  
6     var min_element := v[0];  
7     v[0] := v[High(v)];  
8     SetLength(v, Length(v) - 1);  
9     if not IsEmpty(v) then  
10         SiftDown(v, 0);  
11  
12     Result := min_element;  
13 end;
```

Сложность:

- Перемещение последнего элемента — $O(1)$.
- Уменьшение длины массива — $O(1)$.

- **SiftDown** имеет сложность $O(\log_d(n))$, так как высота d -арного дерева пропорциональна $\log_d(n)$.

Итоговая сложность: $O(\log_d(n))$.

- **IsEmpty** проверяет, пуста ли куча.
Сложность: $O(1)$.
- **SiftDown** восстанавливает свойства кучи, начиная с узла i . Перемещает элемент вниз по дереву, пока он не станет больше своих дочерних элементов.

```

1 procedure SiftDown(var v: array of Integer; index: Integer);
2 var
3     smallest, left, right: Integer;
4 begin
5     smallest := index;
6     left := 2 * index + 1;
7     right := 2 * index + 2;
8
9     if (left < Length(v)) and (v[left] < v[smallest]) then
10         smallest := left;
11
12     if (right < Length(v)) and (v[right] < v[smallest]) then
13         smallest := right;
14
15     if smallest <> index then
16     begin
17         var temp := v[index];
18         v[index] := v[smallest];
19         v[smallest] := temp;
20
21         SiftDown(v, smallest);
22     end;
23 end;
```

Сложность: $O(\log_d(n))$, так как высота дерева пропорциональна $\log_d(n)$.

- **SiftUp** восстанавливает свойства кучи, начиная с узла i . Перемещает элемент вверх по дереву, пока он не станет меньше своих родительских узлов.

```

1 procedure DHeap.SiftUp(i: Integer);
2 var
3     key0, value0: Integer;
4     p: Integer;
5 begin
6     key0 := v_[i].first;
```

```

7      value0 := v_[i].second;
8      p := Father(i);
9
10     while (i <> 0) and (v_[p].second > value0) do
11     begin
12         v_[i] := v_[p];
13         i := p;
14         p := Father(i);
15     end;
16     v_[i] := (key0, value0);
17 end;

```

Сложность: $O(\log_d(n))$, так как высота дерева пропорциональна $\log_d(n)$.

- **MinChild** находит индекс минимального дочернего элемента для узла i . Если у узла нет дочерних элементов, возвращается индекс самого узла.

Сложность: $O(d)$ — требуется проверить d дочерних элементов.

- **LastChild** возвращает индекс последнего дочернего элемента для узла i . Если у узла нет дочерних элементов, возвращается -1.

Сложность: $O(1)$.

- **FirstChild** возвращает индекс первого дочернего элемента для узла i . Если узел не имеет дочерних элементов, возвращается -1.

Сложность: $O(1)$.

- **Father** возвращает индекс родительского узла для узла i .

Сложность: $O(1)$.

3.2 Алгоритм Дейкстры

Генерация графа

Функция **generateGraph** генерирует случайный неориентированный граф с n вершинами и до m ребер. Вес ребер присваиваются случайным образом в диапазоне от q до r .

Максимальное количество ребер ограничено формулой $\frac{n(n-1)}{2}$, которая представляет количество ребер в полном графе. Двумерный вектор используется для отслеживания существующих ребер, чтобы избежать дублирования. Ребра добавляются в список смежности до достижения желаемого количества ребер.

```

1      while edge_count < m do
2      begin
3          i := Random(n) + 1;
4          j := Random(n) + 1;

```

```

5
6     if (i <> j) and (not was[i][j]) then
7     begin
8         weight := Random(r - q + 1) + q;
9         SetLength(adj_list[i], Length(adj_list[i]) + 1);
10        adj_list[i][High(adj_list[i])] . Node := j;
11        adj_list[i][High(adj_list[i])] . Weight := weight;
12
13        SetLength(adj_list[j], Length(adj_list[j]) + 1);
14        adj_list[j][High(adj_list[j])] . Node := i;
15        adj_list[j][High(adj_list[j])] . Weight := weight;
16
17        was[i][j] := True;
18        was[j][i] := True;
19        Inc(edge_count);
20    end;
21 end;

```

Анализ сложности:

- Генерация случайных индексов i и j для вершины выполняется за $O(1)$.
- Проверка уникальности ребра через массив **was** занимает $O(1)$.
- Обновление списка смежности требует $O(1)$ для каждого добавления ребра.

Процесс продолжается, пока не будет создано m уникальных рёбер. Для полного графа ($m = \frac{n(n-1)}{2}$) сложность составляет $O(m)$. С учётом возможных повторов, средняя сложность генерации графа остаётся $O(m)$.

Алгоритм Дейкстры

Функция реализует алгоритм Дейкстры для нахождения кратчайшего пути от стартовой вершины ко всем другим вершинам графа.

Алгоритм инициализирует расстояния до бесконечности (**INT_MAX**) и устанавливает расстояние до стартовой вершины равным нулю. Приоритетная очередь, представляемая (**DHeap**), используется для эффективного извлечения вершины с минимальным расстоянием.

Для каждой соседней вершины проверяется, можно ли улучшить текущее известное расстояние до неё. Если новое расстояние, вычисленное как сумма расстояния до текущей вершины и веса ребра, ведущего к соседней вершине, меньше, чем уже известное расстояние, то оно обновляется. Также в таком случае в кучу добавляется информация о новой вершине с обновленным расстоянием.

Процесс продолжается, пока не будут обработаны все доступные вершины. В итоге алгоритм возвращает массив расстояний, в котором содержатся кратчайшие расстояния от стартовой вершины до всех остальных вершин графа.

```
1  while not dHeap.IsEmpty() do
2  begin
3      minPair := dHeap.ExtractMin();
4      i := minPair.first;
5      currentDist := minPair.second;
6
7      if processed[i] then
8          Continue;
9
10     processed[i] := True;
11
12     for k := 0 to High(graph[i]) do
13     begin
14         j := graph[i][k].first;
15         newDist := graph[i][k].second + currentDist;
16
17         if newDist < dist[j] then
18         begin
19             dist[j] := newDist;
20             up[j] := i;
21             dHeap.Insert(j, dist[j]);
22         end;
23     end;
24 end;
```

Анализ сложности:

- Инициализация массива расстояний: $O(n)$.
- Извлечение вершины из очереди: $O(\log n)$ на операцию.
- Обработка всех соседей вершины: $O(\text{степень вершины})$.
- Обновление расстояний для всех m рёбер требует $O(m \log n)$.

Итоговая сложность алгоритма Дейкстры:

$$O(n \log n + m \log n) = O((n + m) \log n),$$

где n — количество вершин, а m — количество рёбер.

4 Программная реализация алгоритма

4.1 Описание класса DHeap

```
class DHeap {
private:
    int d_;
    vector<pair<int, int>> v_;

    size_t MinChild(size_t i);
    size_t LastChild(size_t i) const;
    size_t FirstChild(size_t i) const;
    size_t Father(size_t i) const;
    void SiftDown(size_t i);
    void SiftUp(size_t i);
public:
    DHeap(int d);
    void Insert(int key, int value);
    pair<int, int> ExtractMin();
    bool IsEmpty() const;
};
```

Поля

- `d_` – количество детей для каждого узла в куче.
- `v_` – вектор пар, хранящий ключи и значения в куче.

Методы

- `DHeap(int d);`
 - **Назначение:** Конструктор, создающий кучу с заданным числом детей для каждого узла.
 - **Входные параметры:** `d` – количество детей для каждого узла.
 - **Выходные параметры:** отсутствуют.
- `void Insert(int key, int value);`
 - **Назначение:** Вставляет новый элемент в кучу.
 - **Входные параметры:** `key` – ключ элемента, `value` – значение элемента.

- **Выходные параметры:** отсутствуют.
- `pair<int, int> ExtractMin();`
 - **Назначение:** Извлекает элемент с минимальным ключом из кучи.
 - **Входные параметры:** отсутствуют.
 - **Выходные параметры:** пара (`key`, `value`) минимального элемента.
- `bool IsEmpty() const;`
 - **Назначение:** Проверяет, пуста ли куча.
 - **Входные параметры:** отсутствуют.
 - **Выходные параметры:** `true`, если куча пуста, иначе `false`.
- `size_t MinChild(size_t i);`
 - **Назначение:** Находит индекс минимального дочернего узла для узла с индексом `i`.
 - **Входные параметры:** `i` – индекс узла.
 - **Выходные параметры:** индекс минимального дочернего узла.
- `size_t LastChild(size_t i) const;`
 - **Назначение:** Находит индекс последнего дочернего узла для узла с индексом `i`.
 - **Входные параметры:** `i` – индекс узла.
 - **Выходные параметры:** индекс последнего дочернего узла.
- `size_t FirstChild(size_t i) const;`
 - **Назначение:** Находит индекс первого дочернего узла для узла с индексом `i`.
 - **Входные параметры:** `i` – индекс узла.
 - **Выходные параметры:** индекс первого дочернего узла.
- `size_t Father(size_t i) const;`
 - **Назначение:** Находит индекс родительского узла для узла с индексом `i`.
 - **Входные параметры:** `i` – индекс узла.
 - **Выходные параметры:** индекс родительского узла.
- `void SiftDown(size_t i);`
 - **Назначение:** Восстанавливает свойства кучи, двигая узел вниз по дереву.

- **Входные параметры:** i – индекс узла.
 - **Выходные параметры:** отсутствуют.
- `void SiftUp(size_t i);`
 - **Назначение:** Восстанавливает свойства кучи, двигая узел вверх по дереву.
 - **Входные параметры:** i – индекс узла.
 - **Выходные параметры:** отсутствуют.

4.2 Описание функций

Функция для создания графа

```
vector<vector<pair<int, int>>> generateGraph(int n, int m, int q, int r) {
    vector<vector<pair<int, int>>> adj_list(n);
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> dist(q, r);
    vector<vector<bool>> was(n, vector<bool>(n, false));

    for (int edge_count = 0; edge_count < min(n * (n - 1) / 2, m);) {
        int i = rand() % n, j = rand() % n;
        if (i != j && !was[i][j]) {
            int weight = dist(gen);
            adj_list[i].push_back({ j, weight });
            adj_list[j].push_back({ i, weight });
            was[i][j] = was[j][i] = true;
            ++edge_count;
        }
    }
    return adj_list;
}
```

Эта функция `generateGraph` предназначена для создания случайного взвешенного графа. Она создает список смежности для графа с заданным количеством узлов и случайным набором рёбер, каждое из которых имеет случайный вес в заданном диапазоне.

Назначение:

Генерация случайного неориентированного графа с определённым числом узлов и рёбер, где веса рёбер выбираются случайным образом в указанном диапазоне. Такой граф

может использоваться для тестирования алгоритмов на больших случайных данных.

Параметры:

- `int n` — количество узлов (вершин) графа.
- `int m` — максимальное количество рёбер.
- `int q` — минимально возможный вес рёбер.
- `int r` — максимально возможный вес рёбер.

Возвращаемое значение:

Возвращает граф в виде списка смежности, представленного вектором векторов пар. Каждая пара в списке смежности содержит узел, с которым соединена вершина, и вес рёбра.

Алгоритм Дейкстры

```
double dijkstra(vector<int>& dist, vector<int>& up,
                vector<vector<pair<int, int>>> graph, int n, int d, int start) {
    vector<bool> processed(n, false);
    fill(dist.begin(), dist.end(), INT_MAX);
    fill(up.begin(), up.end(), -1);
    dist[start] = 0;
    DHeap dHeap(d);
    dHeap.Insert(start, 0);

    auto begin = chrono::high_resolution_clock::now();

    while (!dHeap.IsEmpty()) {
        auto [i, currentDist] = dHeap.ExtractMin();
        if (processed[i]) continue;
        processed[i] = true;

        for (auto& [j, weight] : graph[i]) {
            int newDist = currentDist + weight;
            if (newDist < dist[j]) {
                dist[j] = newDist;
                up[j] = i;
                dHeap.Insert(j, newDist);
            }
        }
    }
}
```

```

        }
    }
}

    auto end = chrono::high_resolution_clock::now();
    return chrono::duration<double>(end - begin).count();
}

```

Функция `dijkstra` реализует алгоритм Дейкстры для нахождения кратчайших путей от заданной вершины до всех остальных. Алгоритм использует специальную структуру данных `DHeap`, которая позволяет ускорить поиск на графах.

Назначение:

Вычисление кратчайших расстояний от начального узла до всех других узлов графа с неотрицательными весами рёбер.

Параметры:

- `vector<int>& dist` — вектор, в который записываются минимальные расстояния от начального узла до всех других узлов.
- `vector<int>& up` — вектор предшественников, где для каждого узла сохраняется его предыдущий узел в кратчайшем пути.
- `vector<vector<pair<int, int>>> graph` — граф, представленный в виде списка смежности.
- `int n` — количество узлов в графе.
- `int d` — степень структуры данных `DHeap`, которая определяет количество детей в каждом узле кучи.
- `int start` — начальный узел, с которого начинается поиск кратчайших путей.

Возвращаемое значение:

Возвращает время выполнения алгоритма в секундах, что позволяет оценить его эффективность на данном графе.

5 Эксперименты

Ниже показаны результаты выполнения алгоритма Дейкстры с использованием 3-кучи и 15-кучи на графах, варьирующихся по количеству вершин и рёбер. Эти эксперименты позволяют оценить влияние структуры кучи на производительность алгоритма при изменении различных параметров графа.

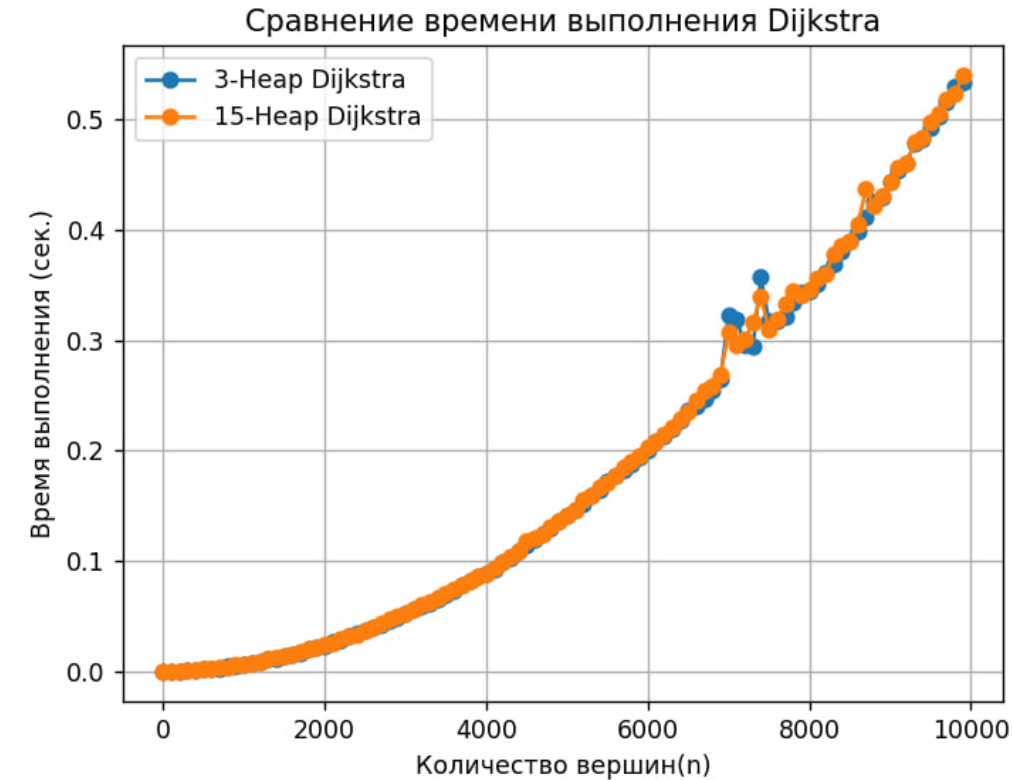


Рис. 1: Сравнение времени выполнения в зависимости от количества вершин

На графике 1 представлена зависимость времени выполнения от количества вершин n . Здесь видно, что использование как 3-кучи, так и 15-кучи приводит к схожим результатам. Различия в скорости выполнения становятся заметны только при больших значениях n , при этом 15-куча начинает слегка уступать 3-куче по скорости выполнения. Это может быть связано с тем, что при увеличении арности увеличивается количество операций управления в более крупных кучах.

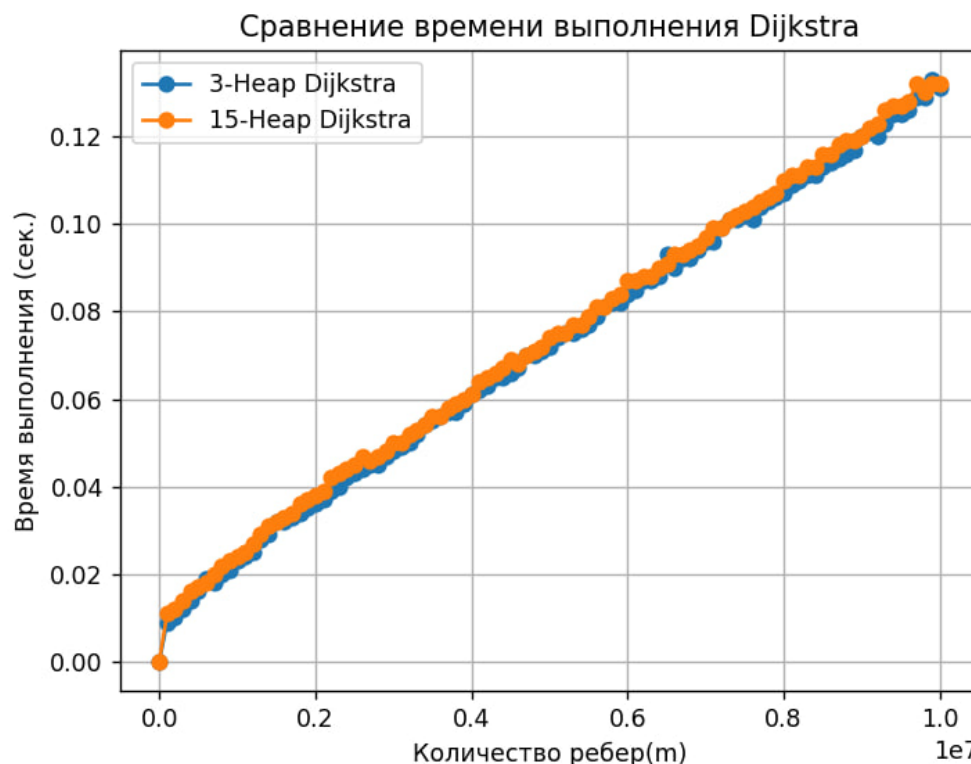


Рис. 2: Сравнение времени выполнения в зависимости от количества рёбер

На графике 2 представлена зависимость времени выполнения от количества рёбер m . В этом случае время выполнения алгоритма также линейно возрастает с увеличением m , и различия между 3-кучей и 15-кучей остаются минимальными. Однако, как и в случае с увеличением вершин, можно заметить, что 15-куча начинает показывать слегка большее время выполнения на больших значениях m . Это связано с тем, что на больших графах операции в 15-куче могут требовать больше времени для поддержания структуры кучи при добавлении и удалении элементов.

Возможные причины отклонений

На графиках также можно заметить незначительные отклонения, которые могут быть обусловлены фоновыми процессами операционной системы, влиянием кэширования и доступом к оперативной памяти. Эти факторы могли вызвать незначительные колебания во времени выполнения, особенно при многократном запуске алгоритма на больших графах.

Описание вычислительной среды

Все эксперименты проводились на вычислительной машине со следующими характеристиками:

- **Процессор:** AMD Ryzen 5 5600X

- **Оперативная память:** 16 GB DDR4
- **Операционная система:** Windows 10 Pro
- **Хранилище:** SSD 512 GB
- **Среда разработки:** Visual Studio Code

Данные параметры были выбраны для обеспечения воспроизводимости экспериментов, а также минимизации влияния аппаратных ограничений на результаты.

Заключение

В ходе исследования был рассмотрен алгоритм Дейкстры с d -кучей при значениях параметра $d = 3$ и $d = 15$. Анализ показал, что при увеличении d существенного улучшения производительности не наблюдается.

1. 3-куча с меньшей арностью позволяет быстрее извлекать минимальный элемент, что дает преимущество на крупных графах.
2. 15-куча с большей арностью снижает высоту структуры, но требует больше операций для поддержания порядка, что может снизить её эффективность при большом количестве вершин и рёбер.

В результате, для большинства задач 3-куча и 15-куча взаимозаменяемы, но 3-куча демонстрирует небольшое преимущество на крупных графах. Увеличение параметра d не оказывает значительного влияния на эффективность алгоритма, что объясняется особенностями самой задачи поиска кратчайших путей.

Список литературы

1. Алгоритм Дейкстры // Википедия: свободная энциклопедия.
URL: https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm.
2. Кормен Т.Х., Лейзерсон Ч.Е., Ривест Р.Л., Штайн К. Алгоритмы: Построение и анализ / Т.Х. Кормен, Ч.Е. Лейзерсон, Р.Л. Ривест, К. Штайн. — 3-е изд. — М.: Вильямс, 2011.
3. Скиен С. Алгоритмы. Руководство по разработке / С. Скиен. — М.: Вильямс, 2006. — 800 с.: ил. ISBN 978-5-8459-0805-7.
4. Алгоритм Дейкстры.
URL: http://www.e-maxx-ru.1gb.ru/algo/dijkstra_sparse.

Приложения

Приложение А. Реализация DHeap

```
1 class DHeap {
2 public:
3     DHeap(int d);
4     void Insert(int key, int value);
5     std::pair<int, int> ExtractMin();
6     bool IsEmpty() const;
7
8 private:
9     size_t MinChild(size_t i);
10    size_t LastChild(size_t i) const;
11    size_t FirstChild(size_t i) const;
12    size_t Father(size_t i) const;
13    void SiftDown(size_t i);
14    void SiftUp(size_t i);
15
16    int d_;
17    std::vector<std::pair<int, int>> v_;
18 };
19
20 size_t DHeap::MinChild(size_t i) {
21     size_t first = FirstChild(i);
22     if (first == -1) return i;
23
24     size_t last = LastChild(i);
25     size_t min_idx = first;
26
27     for (size_t j = first + 1; j <= last; ++j) {
28         if (v_[j].second < v_[min_idx].second) {
29             min_idx = j;
30         }
31     }
32     return min_idx;
33 }
34
35 size_t DHeap::LastChild(size_t i) const {
36     size_t first = FirstChild(i);
37     return (first == -1) ? -1 : std::min(first + d_ - 1, v_.size() - 1);
38 }
39
40 size_t DHeap::FirstChild(size_t i) const {
41     size_t k = d_ * i + 1;
42     return (k < v_.size()) ? k : -1;
```

```

43 }
44
45 size_t DHeap::Father(size_t i) const {
46     return (i - 1) / d_;
47 }
48
49 void DHeap::SiftDown(size_t i) {
50     int key0 = v_[i].first;
51     int value0 = v_[i].second;
52     size_t c = MinChild(i);
53
54     while (c != i && value0 > v_[c].second) {
55         v_[i] = v_[c];
56         i = c;
57         c = MinChild(i);
58     }
59     v_[i] = { key0, value0 };
60 }
61
62 void DHeap::SiftUp(size_t i) {
63     int key0 = v_[i].first;
64     int value0 = v_[i].second;
65     size_t p = Father(i);
66
67     while (i != 0 && v_[p].second > value0) {
68         v_[i] = v_[p];
69         i = p;
70         p = Father(i);
71     }
72     v_[i] = { key0, value0 };
73 }
74
75 DHeap::DHeap(int d) : d_(d) {
76     if (d_ < 2) throw std::runtime_error("Degree of heap must be at least 2.");
77 }
78
79 void DHeap::Insert(int key, int value) {
80     v_.push_back({ key, value });
81     SiftUp(v_.size() - 1);
82 }
83
84 std::pair<int, int> DHeap::ExtractMin() {
85     if (v_.empty()) throw std::runtime_error("Heap is empty");
86
87     std::pair<int, int> min_element(v_[0]);
88     v_[0] = v_.back();

```

```

89     v_.pop_back();
90     if (!v_.empty()) SiftDown(0);
91
92     return min_element;
93 }
94
95 bool DHeap::IsEmpty() const {
96     return v_.empty();
97 }

```

Приложение Б. Реализация Дейкстры

```

1  vector<vector<pair<int, int>>> generateGraph(int n, int m, int q, int r) {
2      m = min(n * (n - 1) / 2, m);
3      vector<vector<pair<int, int>>> adj_list(n);
4
5      std::random_device rd;
6      std::mt19937 gen(rd());
7      std::uniform_int_distribution<> dist(q, r);
8
9      vector<vector<bool>> was(n, vector<bool>(n, false));
10     int edge_count = 0;
11
12     while (edge_count < m) {
13         int i = rand() % n;
14         int j = rand() % n;
15         if (i != j && !was[i][j]) {
16             int weight = dist(gen);
17
18             adj_list[i].push_back({ j, weight });
19             adj_list[j].push_back({ i, weight });
20
21             was[i][j] = was[j][i] = true;
22             ++edge_count;
23         }
24     }
25     return adj_list;
26 }
27
28 double dijkstra(vector<int>& dist, vector<int>& up, vector<vector<pair<int,
29     int>>> graph, int n, int d, int start) {
30     vector<bool> processed(n, false);
31     vector<int> index(n);
32     vector<int> name(n);

```

```

33     for (int i = 0; i < n; ++i) {
34         up[i] = -1;
35         dist[i] = INT_MAX;
36         index[i] = i;
37         name[i] = i;
38     }
39
40     dist[start] = 0;
41     up[start] = start;
42     DHeap dHeap(d);
43     dHeap.Insert(start, dist[start]);
44
45     auto begin = std::chrono::high_resolution_clock::now();
46
47     while (!dHeap.IsEmpty()) {
48         pair<int, int> minPair = dHeap.ExtractMin();
49         int i = minPair.first;
50         int currentDist = minPair.second;
51
52         if (processed[i]) continue;
53         processed[i] = true;
54
55         vector<pair<int, int>> edge = graph[i];
56         for (int k = 0; k < edge.size(); k++) {
57             int j = edge[k].first;
58             int newDist = edge[k].second + currentDist;
59
60             if (newDist < dist[j]) {
61                 dist[j] = newDist;
62                 up[j] = i;
63                 dHeap.Insert(j, dist[j]);
64             }
65         }
66     }
67
68     auto end = std::chrono::high_resolution_clock::now();
69     std::chrono::duration<double> executionTime = end - begin;
70     std::cout << "Execution time: " << executionTime.count() << " seconds\n"
71     ;
72     return executionTime.count();
73 }

```